HO CHI MINH CITY UNIVERSITY OF TRANSPORT

Kiến thức - Kỹ năng - Sáng tạo - Hội nhập

Sứ mệnh - Tầm nhìn

Triết lý Giáo dục - Giá trị cốt lõi

"For the things we have to learn before we can do them, we learn by doing them."
— Aristotle, *The Nicomachean Ethics* (Book II) —

Contents

0 Library Resources **3**

0.1 Search Agents . . . 3

0.2 Constraints & Knowledge (Prolog programming Language) . . . 3

0.3 Machine Learning . . . 3

0.4 Neural Networks . . . 3

0.5 Data Mining . . . 3

1 Intelligence Agents **4**

2 Search Agents **4**

3 Constraint Satisfaction Problem (CSP) **7**

4 Knowledge - Proposition Logic **11**

5 Supervised Learning **16**

5.1 Logistic Regression . . . 17

6 Naive Bayes Classifier **17**

7 Decision Tree **18**

8 Nonparametric Models **19**

8.1 K-Nearest Neighbors (KNN) . . . 19

8.2 Support Vector Machine . . . 20

9 Ensemble Learning **21**

9.1 Wisdom of the Crowd . . . 21

9.2 Random Forest . . . 22

10 Unsupervised Learning **22**

10.1 K-Means Algorithm . . . 22

10.2 Principal Component Analysis . . . 22

11 Neural Networks **23**

11.1 Rosenblatt's Perceptron 23

11.2 Multilayer Networks 24

11.3 Backward Propagation of Errors 24

0 Library Resources

0.1 Search Agents

- https://networkx.org/

0.2 Constraints & Knowledge (Prolog programming Language)

SWI stand for **Sociaal-Wetenschappelijke Informatica** (Dutch).

- Install on PC for Coding with VS Code:
  https://www.swi-prolog.org/download/stable
- SWISH (SWI-Prolog for Web) - Web base coding:
  https://swish.swi-prolog.org/
- TutorialsPoint Prolog Terminal - Web base coding:
  https://www.tutorialspoint.com/execute_prolog_online.php
- Wumpus World Game:
  https://github.com/nvdieu/wumpus-prolog-game

0.3 Machine Learning

- https://developers.google.com/machine-learning/crash-course
- https://www.kaggle.com/

0.4 Neural Networks

- https://developers.google.com/machine-learning/crash-course/neural-networks
- https://alexlenail.me/NN-SVG/index.html
- https://playground.tensorflow.org/
- http://neuralnetworksanddeeplearning.com/

0.5 Data Mining

- https://orangedatamining.com/
- https://www.knime.com/
- https://www.weka.io/

1 Intelligence Agents

1.1 Đặc tả PEAS cho Agent thực tế

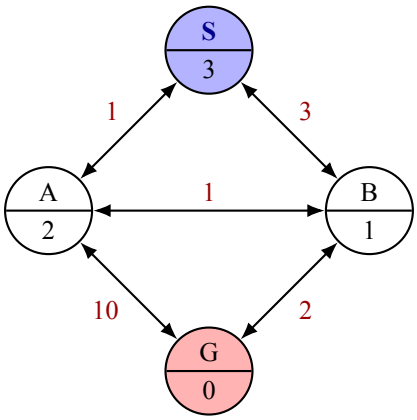
- **Yêu cầu:** Hãy mô tả mô hình PEAS cho các Intelligent Agents sau:
  1. Một Agent hỗ trợ phẫu thuật y tế từ xa.
  2. Một hệ thống tự động phân loại rác thải trên băng chuyền.
  3. Một Agent tối ưu hóa giá vé máy bay theo thời gian thực.

1.2 Phân loại môi trường (Task Environment)

- **Yêu cầu:** Điền vào bảng phân loại môi trường cho các tác vụ sau:
  1. Chơi cờ vây: (Quan sát? Ngẫu nhiên? Thời gian? Tác tử?)
  2. Mua sắm trực tuyến: (Quan sát? Ngẫu nhiên? Thời gian? Tác tử?)

2 Search Agents

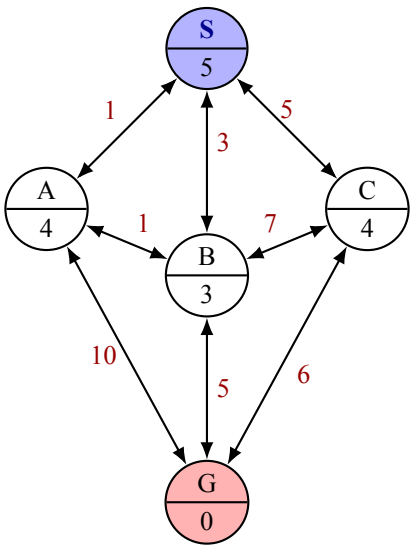
2.1 Searching



Initial state: **S**; Goal state: **G**

a) Best-first search.
b) Greedy Best-first search.
c) A* search.

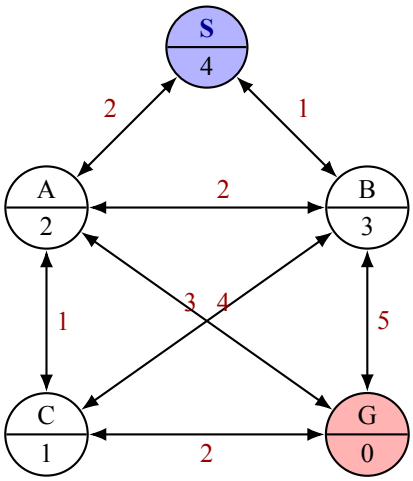
2.2 Searching



Initial state: **S**; Goal state: **G**

a) Best-first search.
b) Greedy Best-first search.
c) A* search.

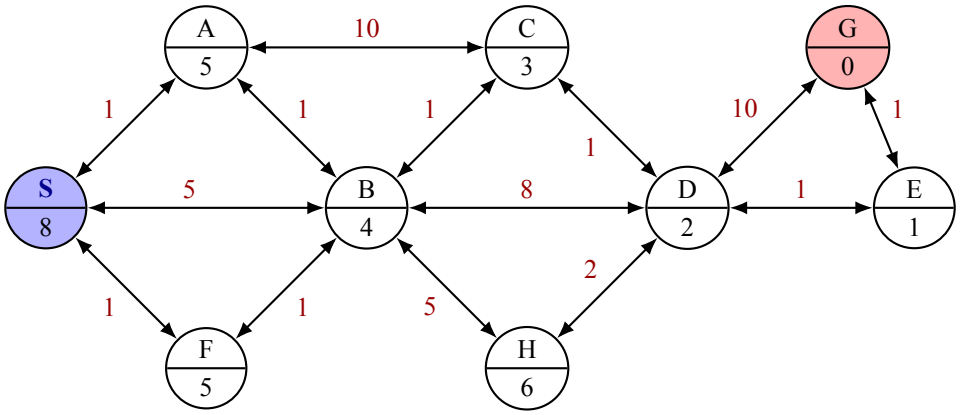
2.3 Searching



Initial state: **S**; Goal state: **G**

a) Best-first search.
b) Greedy Best-first search.
c) A* search.

2.4 Searching (Turtle)

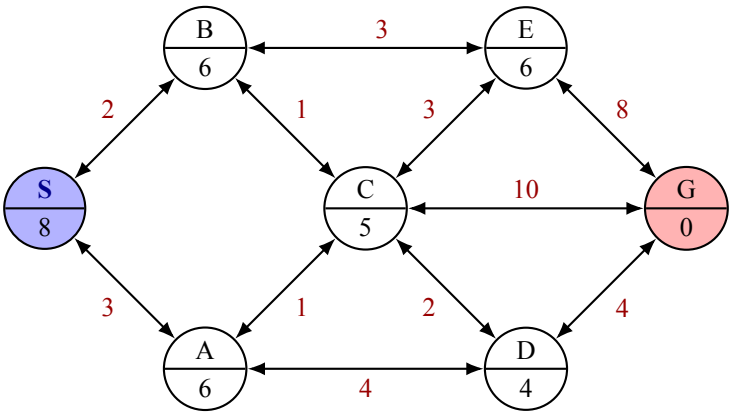


Turtle

Initial state: **S**; Goal state: **G**

a) Best-first search.
b) Greedy Best-first search.
c) A* search.

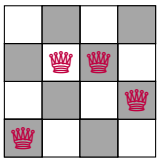
2.5 Searching



Initial state: **S**; Goal state: **G**

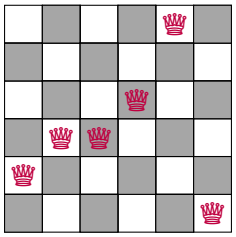
a) Best-first search.
b) Greedy Best-first search.
c) A* search.

2.6 4-Queens



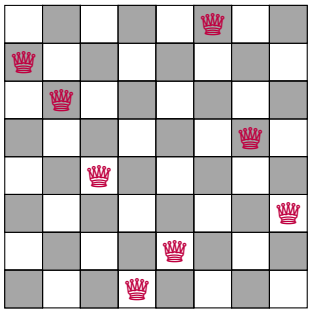
- h = The number of pairs of queens that are Conflicts each other.
- Hill-climbing Search.

2.7 6-Queens



- h = The number of pairs of queens that are Conflicts each other.
- Hill-climbing Search.

2.8 8-Queens



- h = The number of pairs of queens that are Conflicts each other.
- Hill-climbing Search.

2.9 The 8-puzzle problem

Initial start:

2	1	3
8		4
7	6	5

Goal start:

1	2	3
4	5	6
7	8	

h_1**: Number of misplaced tiles**.
h_2**: Sum of Manhattan distances of the tiles from their goal positions**.

Draw tree searching by use:

a) Greedy Best-first with h_1, h_2.
b) A* with h_1, h_2.

2.10 The 8-puzzle problem

Initial start:

1	2	3
8		4
7	5	6

Goal start:

1	2	3
4	5	6
7	8	

h_1**: Number of misplaced tiles**.
h_2**: Sum of Manhattan distances of the tiles from their goal positions**.

Draw tree searching by use:

a) Greedy Best-first with h_1, h_2.
b) A* with h_1, h_2.

3 Constraint Satisfaction Problem (CSP)

3.1 Exam Scheduling Problem (AC-3)

Given a set of 4 subjects $\{M_1, M_2, M_3, M_4\}$ to be scheduled into 3 available time slots $\{1, 2, 3\}$.

- **Variables (X):** M_1, M_2, M_3, M_4
- **Domains (D):**
  - $D_{M_1} = \{1, 2, 3\}$
  - $D_{M_2} = \{1, 2, 3\}$
  - $D_{M_3} = \{1, 2, 3\}$
  - $D_{M_4} = \{1, 2\}$
- **Constraints (C):**
  1. $M_1 \neq M_2$ (Subject 1 and 2 cannot be in the same slot).
  2. $M_1 < M_3$ (Subject 1 must be scheduled before Subject 3).
  3. $M_2 = M_3$ (Subject 2 and 3 must be in the same slot).

4. $|M_3 - M_4| \geq 2$ (Subject 3 and 4 must be at least 2 slots apart).

Requirements:

1. Draw the Constraint Graph.
2. Trace the AC-3 algorithm.

3.2 Resource Allocation Problem (AC-3)

An investment firm is allocating capital (in billions of VND) to 3 projects X, Y, and Z.

- **Variables (X):** X, Y, Z
- **Domains (D):**
  - $D_X = \{1, 2, 3, 4, 5, 6\}$
  - $D_Y = \{2, 4, 6, 9\}$
  - $D_Z = \{1, 4, 9, 10\}$
- **Constraints (C):**
  1. $X + Y = 6$ (Total investment for X and Y must equal 6).
  2. $Y < Z$ (Project Y must receive less than to the capital of Project Z).
  3. $Z = X^2$ (Capital for Z must be the square of capital for X).

  (Another case: $Y \leq Z$)

Requirements:

1. Draw the Constraint Graph.
2. AC-3 algorithm.

3.3 Einstein's Riddle (The Zebra Puzzle)

1. The Situation

Có 5 ngôi nhà với 5 màu sắc khác nhau. Trong mỗi nhà có một người với quốc tịch khác nhau. 5 chủ nhà này uống các loại đồ uống khác nhau, hút các loại thuốc lá khác nhau và nuôi các loại thú cưng khác nhau. Không có ai có cùng thú cưng, hút cùng loại thuốc hay uống cùng một loại đồ uống.

2. The 15 Clues

1. The Brit lives in the Red house. (Người Anh ở nhà đỏ).
2. The Spaniard owns the Dog. (Người Tây Ban Nha nuôi chó).
3. Coffee is drunk in the Green house. (Cà phê được uống ở nhà xanh lá).
4. The Ukrainian drinks Tea. (Người Ukraine uống trà).
5. The Green house is immediately to the right of the Ivory house. (Nhà xanh lá nằm ngay bên phải nhà màu ngà).
6. The Old Gold smoker owns Snails. (Người hút thuốc Old Gold nuôi ốc sên).
7. Kools are smoked in the Yellow house. (Thuốc Kools được hút ở nhà vàng).
8. Milk is drunk in the middle house. (Sữa được uống ở nhà chính giữa - nhà số 3).
9. The Norwegian lives in the first house. (Người Na Uy ở nhà đầu tiên - nhà số 1).
10. The man who smokes Chesterfields lives in the house next to the man with the Fox. (Người hút Chesterfields ở cạnh nhà người nuôi cáo).
11. Kools are smoked in the house next to the house where the Horse is kept. (Thuốc Kools được hút ở cạnh nhà nuôi ngựa).
12. The Lucky Strike smoker drinks Orange Juice. (Người hút Lucky Strike uống nước cam).
13. The Japanese smokes Parliaments. (Người Nhật hút thuốc Parliaments).
14. The Norwegian lives next to the Blue house. (Người Na Uy ở cạnh nhà màu xanh lam).

15. The man who smokes Chesterfields has a neighbor who drinks Water. (Người hút Chesterfields có hàng xóm uống nước).

3. Question: Who owns the Zebra? Who drinks Water?

Requirements:

- **AC-3**
- **Backtracking**

Gợi ý:

Variables & Domains
Ta có 5 biến cho mỗi loại thuộc tính (tổng cộng 25 biến):

- H_1, H_2, H_3, H_4, H_5 (Thứ tự nhà từ trái sang phải).
- Mỗi nhà có các thuộc tính: Color, Nationality, Drink, Tobacco, Pet.
- Miền giá trị cho mỗi thuộc tính là tập hợp 5 lựa chọn (ví dụ $D_{Color} = \{Red, Green, Ivory, Yellow, Blue\}$).

Types of Constraints

1. **Unary Constraints**
   - Clue 8: $Drink_3 = Milk$ (Nhà giữa uống sữa).
   - Clue 9: $Nationality_1 = Norwegian$ (Nhà đầu là người Na Uy).
2. **Binary Constraints** Nếu thuộc tính X của nhà i là a thì thuộc tính Y của nhà i phải là b:
   - Clue 1: $(Nationality_i = Brit) \iff (Color_i = Red)$
   - Clue 2: $(Nationality_i = Spanish) \iff (Pet_i = Dog)$
   - Clue 3: $(Color_i = Green) \iff (Drink_i = Coffee)$
   - Clue 4: $(Nationality_i = Ukrainian) \iff (Drink_i = Tea)$
   - Clue 6: $(Tobacco_i = Old_Gold) \iff (Pet_i = Snails)$
   - Clue 7: $(Tobacco_i = Kools) \iff (Color_i = Yellow)$
   - Clue 12: $(Tobacco_i = Lucky_Strike) \iff (Drink_i = Orange_Juice)$
   - Clue 13: $(Nationality_i = Japanese) \iff (Tobacco_i = Parliaments)$
3. **Relative Constraints**
   - Clue 5 (Bên phải trực tiếp):
     $(Color_i = Ivory) \iff (Color_{i+1} = Green)$ (với $i \in \{1, 2, 3, 4\}$).
   - Clue 10 (Kế bên nhau - Next to):
     $(Tobacco_i = Chesterfields) \iff (Pet_{i-1} = Fox \lor Pet_{i+1} = Fox)$
   - Clue 11 (Kế bên nhau):
     $(Tobacco_i = Kools) \iff (Pet_{i-1} = Horse \lor Pet_{i+1} = Horse)$
   - Clue 14 (Kế bên nhau):
     $(Nationality_i = Norwegian) \iff (Color_{i-1} = Blue \lor Color_{i+1} = Blue)$
   - Clue 15 (Kế bên nhau):
     $(Tobacco_i = Chesterfields) \iff (Drink_{i-1} = Water \lor Drink_{i+1} = Water)$
4. **Implicit Constraints (Ràng buộc ẩn)**
   All-Different: Tất cả 5 giá trị trong cùng một hạng mục (ví dụ 5 quốc tịch) phải khác nhau:
   $$\forall i, j \in \{1..5\}, i \neq j \implies Nationality_i \neq Nationality_j$$

3.4 Scheduling Classes

Let's consider a scheduling problem for a university with multiple courses and classrooms. The goal is to schedule classes in a way that satisfies various constraints:

Variables:

- Courses: Math (M), Physics (P), Chemistry (C), Biology (B)

- Classrooms: R1, R2, R3

Constraints:

1. Each course must be assigned to exactly one classroom.
2. No two courses can be held in the same classroom at the same time.
3. There are specific time constraints for certain courses:
    - Math and Physics cannot be scheduled at the same time.
    - Chemistry and Biology cannot be scheduled at the same time.
4. Each course has preferred time slots:
    - Math prefers morning sessions.
    - Physics prefers afternoon sessions.
    - Chemistry prefers morning sessions.
    - Biology prefers afternoon sessions.

3.5 Scheduling problem

You're tasked with scheduling a series of events at a conference. The conference has multiple sessions and each session can accommodate different talks. However, there are various constraints and preferences to consider:

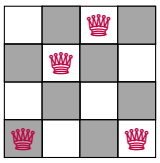
Variables:

- Talks: A, B, C, D, E, F
- Sessions: Morning Session (M), Afternoon Session (A), Evening Session (E)

Constraints:

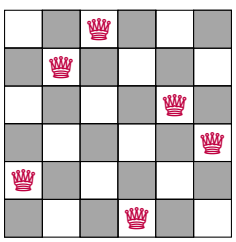
1. Each talk must be assigned to exactly one session.
2. No two talks can be scheduled in the same session simultaneously.
3. Certain talks have special requirements:
    - Talk A and B are co-dependent and must be scheduled in the same session.
    - Talk C and D cannot be scheduled in the same session.
    - Talk E must be scheduled before talk F.

3.6 4-Queens



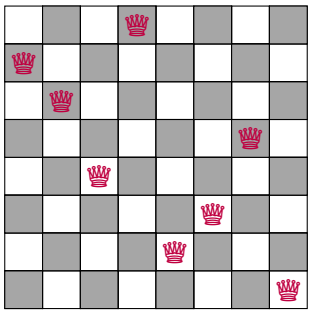
- h = The number of pairs of queens that are Conflicts each other.
- Local search with Min-Conflicts Heuristic.

3.7 6-Queens



- h = The number of pairs of queens that are Conflicts each other.
- Local search with Min-Conflicts Heuristic.

3.8 8-Queens



- h = The number of pairs of queens that are Conflicts each other.
- Local search with Min-Conflicts Heuristic.

3.9 Wolf, Goat and Cabbage problem

Một nông dân có 3 mặt hàng: một con sói, một con dê và một quả bắp cải phải băng qua sông bằng thuyền.
Thuyền chỉ có thể mang theo nông dân và một mặt hàng duy nhất.
Nếu ở cùng nhau và không có người nông dân thì sói sẽ ăn dê, hoặc dê sẽ ăn bắp cải.
Làm thế nào họ có thể qua sông mà không có gì bị ăn?

Lưu ý: Đây là bài toán cho cả CSP và Proposition Logic.

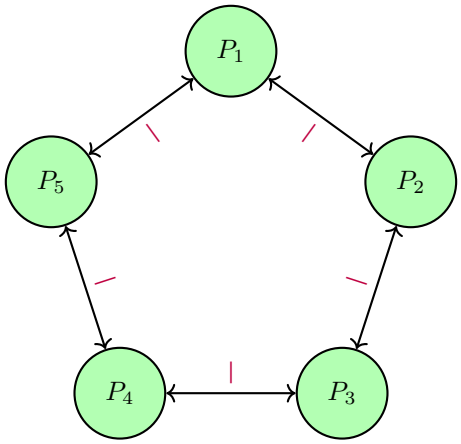
3.10 The Dining Philosophers Problem

Có n nhà triết học ngồi quanh một chiếc bàn tròn. Giữa mỗi cặp nhà triết học có một chiếc đũa (tổng cộng có n chiếc đũa).

- Mỗi nhà triết học chỉ làm hai việc: Suy nghĩ hoặc Ăn.
- Để ăn, một nhà triết học cần có cả hai chiếc đũa (bên trái và bên phải).
- Sau khi ăn xong, họ đặt đũa xuống và tiếp tục suy nghĩ.

CSP

- **Variable:** $P_1, P_2, \ldots, P_n$ Trạng thái của từng nhà triết học.
- **Domain:** $D = \{Thinking, Eating\}$
- **Constraint:** $\neg(P_i = Eating \land P_{i+1} = Eating)$ Hai nhà triết học ngồi cạnh nhau không thể cùng ở trạng thái $Eating$ vì họ dùng chung một chiếc đũa.
- **Required:** Tìm một lời giải hoàn chỉnh (Complete Assignment) thỏa mãn tất cả ràng buộc.



4 Knowledge - Proposition Logic

4.1 Compute the Truth tables

a) $(\neg P) \Leftrightarrow (\neg Q)$ $(compare\ to\ P \Leftrightarrow Q)$
b) $[P \vee (\neg Q)] \Leftrightarrow P$
c) $\neg[P \wedge (Q \vee R)]$
d) $\neg(P \wedge Q)$
e) $P \wedge (\neg P)) \Rightarrow Q$
f) $(P \wedge Q) \Rightarrow P$
g) $P \Rightarrow (P \vee Q)$

4.2 Truth tables method

a) $(P \Rightarrow Q) \vee (P \Rightarrow \neg Q)$: **valid?**
b) $(\neg P \vee Q) \wedge (Q \Rightarrow \neg R \wedge \neg P) \wedge (P \vee R)$: **satisfiable?**
c) Determine whether the $(P \wedge \neg Q \Rightarrow P \wedge Q)$ is a **logical consequence** of the $\neg P$
d) Determine whether $P \Rightarrow (Q \wedge \neg Q)$ and $\neg P$ are **logically equivalent**

4.3 Truth tables method verify whether the following logical consequences and equivalences are correct

a) $(P \Rightarrow Q) \models \neg P \Rightarrow \neg Q$
b) $(P \Rightarrow Q) \wedge \neg Q \models \neg P$
c) $P \Rightarrow (Q \wedge R) \models (P \Rightarrow Q) \Rightarrow R$
d) $P \vee (\neg Q \wedge R) \models Q \vee \neg R \Rightarrow P$
e) $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$
f) $(P \vee Q) \wedge (\neg P \Rightarrow \neg Q) \equiv Q$
g) $(P \wedge Q) \vee R \models (P \Rightarrow \neg Q) \Rightarrow R$
h) $(P \vee Q) \wedge (\neg P \Rightarrow \neg Q) \equiv P$
i) $((P \Rightarrow Q) \Rightarrow Q) \Rightarrow Q \equiv P \Rightarrow Q$

4.4 Which of the following are correct?

a) $False \models True$
b) $True \models False$
c) $(A \wedge B) \models (A \Leftrightarrow B)$
d) $A \Leftrightarrow B \models A \vee B$
e) $A \Leftrightarrow B \models \neg A \vee B$
f) $(A \wedge B) \Rightarrow C \models (A \Rightarrow C) \vee (B \Rightarrow C)$
g) $(C \vee (\neg A \wedge \neg B)) \equiv ((A \Rightarrow C) \wedge (B \Rightarrow C))$
h) $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B)$
i) $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B) \wedge (\neg D \vee E)$
j) $(A \vee B) \wedge \neg(A \rightarrow B)$ is satisfiable.
k) $(A \Leftrightarrow B) \wedge (\neg A \vee B)$ is satisfiable.
l) $(A \Leftrightarrow B) \Leftrightarrow C$ has the same number of models as $(A \Leftrightarrow B)$
for any fixed set of proposition symbols that includes A, B, C.

4.5 Prove each of the following assertions:

a) α is valid if and only if $True \models \alpha$
b) For any α, $False \models \alpha$
c) $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid.
d) $\alpha \equiv \beta$ if and only if the sentence $(\alpha \Leftrightarrow \beta)$ is valid.
e) $\alpha \models \beta$ if and only if the sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable.

4.6 Prove, or find a counterexample to, each of the following assertions:

a) If $\alpha \models \gamma$ or $\beta \models \gamma$ (or both) then $(\alpha \wedge \beta) \models \gamma$
b) If $(\alpha \wedge \beta) \models \gamma$ then $\alpha \models \gamma$ or $\beta \models \gamma$ (or both).
c) If $\alpha \models (\beta \vee \gamma)$ then $\alpha \models \beta$ or $\alpha \models \gamma$ (or both).
d) If $\alpha \models (\beta \wedge \gamma)$ then $\alpha \models \beta$ and $\alpha \models \gamma$

4.7 Valid, unsatisfiable, or neither

Verify your decisions using truth tables or the equivalence rules of Table.

a) $Smoke \Rightarrow Smoke$
b) $Smoke \Rightarrow Fire$
c) $(Smoke \Rightarrow Fire) \Rightarrow (\neg Smoke \Rightarrow \neg Fire)$
d) $Smoke \vee Fire \vee \neg Fire$
e) $((Smoke \wedge Heat) \models Fire) \Leftrightarrow ((Smoke \models Fire) \vee (Heat \Rightarrow Fire))$
f) $(Smoke \Rightarrow Fire) \Rightarrow ((Smoke \wedge Heat) \models Fire)$
g) $Big \vee Dumb \vee (Big \Rightarrow Dumb)$

4.8 Convert into CNF

a) $P \Leftrightarrow Q$
b) $(P \Rightarrow Q) \Rightarrow R$
c) $(P \vee Q) \wedge (P \wedge R)$
d) $(P \Rightarrow Q) \Leftrightarrow (P \Rightarrow R)$
e) $(A \wedge \neg\neg B) \vee (\neg C \wedge D)$
f) $(P \vee (Q \wedge \neg A)) \wedge (\neg P \vee A \vee \neg B)$
g) $\neg(P \Rightarrow Q) \vee ((R \vee S) \Rightarrow (Q \vee T)) \vee (\neg P \Rightarrow \neg V)$
h) $P \Rightarrow (Q \wedge R)$
i) $(P \vee Q) \Rightarrow R$
j) $\neg(\neg P \vee Q) \vee (R \Rightarrow \neg S)$
k) $\neg((P \Rightarrow (Q \Rightarrow R))) \Rightarrow ((P \Rightarrow Q) \Rightarrow (P \Rightarrow R))$
l) $P \vee (\neg Q \wedge (R \Rightarrow \neg P))$
m) $\neg((((A \Rightarrow B)) \Rightarrow A) \Rightarrow A)$
n) $\neg(A \vee (A \Rightarrow B))$
o) $\neg P \wedge Q \vee \neg(\neg R \vee \neg Q)$

4.9 Prove with resolution that the following formulas are tautologies

a) $P \Rightarrow (Q \Rightarrow P)$
b) $(P \wedge (P \Rightarrow Q)) \Rightarrow Q$
c) $((P \Rightarrow Q) \wedge \neg Q) \Rightarrow \neg P$
d) $((P \Rightarrow Q) \wedge \neg Q) \Rightarrow \neg Q$

4.10 Prove using resolution that

$P \Rightarrow (Q \wedge R) \models (\neg Q \vee \neg R) \Rightarrow \neg P$

4.11 Prove using resolution that

$P \Rightarrow Q$ is a logical consequence of $((T \Rightarrow Q) \wedge (\neg R \Rightarrow \neg S) \wedge (P \Rightarrow U) \wedge (\neg T \Rightarrow \neg R) \wedge (U \Rightarrow S))$

4.12 Prove with resolution

a) 1. $(P \Rightarrow Q) \Rightarrow Q$
   2. $(P \Rightarrow P) \Rightarrow R$
   3. $(R \Rightarrow S) \Rightarrow \neg(S \Rightarrow Q)$
   $\therefore \ R$

b) 1. $\neg P \vee Q$
   2. $\neg R \Rightarrow \neg Q$
   $\therefore \ P \Rightarrow R$

c) 1. $(P \Rightarrow Q) \Rightarrow Q$
   2. $(P \Rightarrow P) \Rightarrow R$
   3. $(R \Rightarrow S) \Rightarrow \neg(S \Rightarrow Q)$
   $\therefore \ R$

4.13 Forward chaining and Backward chaining

a) 1. $G \Rightarrow A$
   2. $(A \wedge B \wedge C) \Rightarrow D$
   3. $(E \wedge B) \Rightarrow C$
   4. $D \Rightarrow H$
   5. $(C \wedge H) \Rightarrow F$
   6. $G \Rightarrow B$
   7. G
   8. E
   $\therefore \ H$

4.14 The Lady or the Tiger

Frank Stockton's story:

Nhà vua muốn tha thứ cho một tù nhân thông minh nên đề ra trò chơi: Trò chơi có một số phòng và anh tù nhân đến cửa phòng và dựa trên các biển báo của các phòng để mở cửa phòng. Tùy theo luật chơi, nếu tù nhân mở cửa phòng của công chúa, anh ta sẽ được tha tội và được cưới công chúa, nhưng nếu có hổ trong phòng, anh ta sẽ bị hổ ăn thịt.

a) Phiên bản 1 (**2 phòng**).
   • Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.
   • Biển báo trên cửa phòng này đúng thì biển báo trên cửa phòng kia sai và ngược lại.
   • Biển báo trên cửa phòng 1: Phòng này có công chúa và phòng kia có hổ.
   • Biển báo trên cửa phòng 2: Một trong hai phòng này có công chúa và phòng còn lại có hổ.
   Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 2 (**2 phòng**).
   • Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.
   • Cả hai biển báo đều đúng hoặc đều sai.
   • Biển báo trên cửa phòng 1: Ít nhất có một phòng có công chúa.
   • Biển báo trên cửa phòng 2: Phòng kia có hổ.
   Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 3 (**2 phòng**).
   • Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.
   • Cả hai biển báo đều đúng hoặc đều sai.
   • Biển báo trên cửa phòng 1: Hổ ở phòng này hay công chúa ở phòng kia.
   • Biển báo trên cửa phòng 2: Công chúa ở phòng kia.
   Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 4 (**2 phòng**).

• Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.

• Ở phòng 1: Nếu có công chúa thì biển báo của phòng là **đúng** nhưng nếu có hổ trong phòng thì biển báo của phòng là **sai**.

• Ở phòng 2: Ngược lại với phòng 1, nếu có công chúa thì biển báo của phòng là **sai** nhưng nếu có hổ trong phòng thì biển báo của phòng là **đúng**.

• Biển báo trên cửa phòng 1: Cả 2 phòng đều có công chúa.

• Biển báo trên cửa phòng 2: Cả 2 phòng đều có công chúa.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 5 (**2 phòng**).

• Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.

• Ở phòng 1: Nếu có công chúa thì biển báo của phòng là **đúng** nhưng nếu có hổ trong phòng thì biển báo của phòng là **sai**.

• Ở phòng 2: Ngược lại với phòng 1, nếu có công chúa thì biển báo của phòng là **sai** nhưng nếu có hổ trong phòng thì biển báo của phòng là **đúng**.

• Biển báo trên cửa phòng 1: Có ít nhất 1 phòng có công chúa.

• Biển báo trên cửa phòng 2: Phòng kia có công chúa.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 6 (**2 phòng**).

• Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.

• Ở phòng 1: Nếu có công chúa thì biển báo của phòng là **đúng** nhưng nếu có hổ trong phòng thì biển báo của phòng là **sai**.

• Ở phòng 2: Ngược lại với phòng 1, nếu có công chúa thì biển báo của phòng là **sai** nhưng nếu có hổ trong phòng thì biển báo của phòng là **đúng**.

• Biển báo trên cửa phòng 1: Không có sự khác biệt giữa 2 phòng.

• Biển báo trên cửa phòng 2: Phòng kia có công chúa.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 7 (**2 phòng**).

• Có 2 phòng; Mỗi phòng hoặc là có công chúa hoặc là chứa hổ. Có thể cả hai phòng đều chứa hổ hay cả hai phòng đều có công chúa và có thể có một phòng có công chúa, một phòng chứa hổ.

• Ở phòng 1: Nếu có công chúa thì biển báo của phòng là **đúng** nhưng nếu có hổ trong phòng thì biển báo của phòng là **sai**.

• Ở phòng 2: Ngược lại với phòng 1, nếu có công chúa thì biển báo của phòng là **sai** nhưng nếu có hổ trong phòng thì biển báo của phòng là **đúng**.

• Biển báo trên cửa phòng 1: Có sự khác biệt giữa 2 phòng.

• Biển báo trên cửa phòng 2: Bạn nên chọn phòng khác thì tốt hơn.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 8 (**3 phòng**).

• Có 3 phòng; Một phòng có công chúa còn 2 phòng kia chứa hổ.

• Có nhiều nhất 1 phòng có biển báo đúng.

• Biển báo trên cửa phòng 1: Hổ có trong phòng này.

• Biển báo trên cửa phòng 2: Công chúa ở phòng này.

• Biển báo trên cửa phòng 3: Hổ ở phòng số 2.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 9 (**3 phòng**).

• Có 3 phòng; Một phòng có công chúa còn 2 phòng kia chứa hổ.

• Biển báo ở phòng có công chúa là đúng và có ít nhất 1 biển báo của 2 phòng còn lại là sai.

• Biển báo trên cửa phòng 1: Hổ có trong phòng số 2.

• Biển báo trên cửa phòng 2: Hổ có trong phòng này.

• Biển báo trên cửa phòng 3: Hổ ở phòng số 1.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 10 (**3 phòng**).

• Có 3 phòng; Một phòng có công chúa, một phòng khác có hổ và phòng còn lại thì trống.

• Biển báo ở phòng có công chúa là đúng; Biển báo ở phòng có hổ là sai và biển báo ở phòng trống thì có thể đúng hoặc sai.

• Biển báo trên cửa phòng 1: Phòng số 3 trống.

• Biển báo trên cửa phòng 2: Hổ có trong phòng số 1.

• Biển báo trên cửa phòng 3: Phòng này trống.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

a) Phiên bản 11 (**9 phòng**).

• Có 9 phòng; Một phòng có công chúa, các phòng khác có thể có hổ hoặc trống.

• Biển báo ở phòng có công chúa là đúng; Biển báo ở phòng có hổ là sai và biển báo ở phòng trống thì có thể đúng hoặc sai.

• Biển báo trên cửa phòng 1: Công chúa ở phòng số lẻ.

• Biển báo trên cửa phòng 2: Phòng này trống.

• Biển báo trên cửa phòng 3: Biển báo ở phòng số 5 là đúng hay biển báo phòng số 7 là sai.

• Biển báo trên cửa phòng 4: Biển báo ở phòng số 1 là sai.

• Biển báo trên cửa phòng 5: Biển báo ở phòng số 2 hay biển báo ở phòng số 4 là đúng.

• Biển báo trên cửa phòng 6: Biển báo ở phòng số 3 là sai.

• Biển báo trên cửa phòng 7: Công chúa không có trong phòng số 1.

• Biển báo trên cửa phòng 8: Phòng này chứa hổ và phòng số 9 thì trống.

• Biển báo trên cửa phòng 9: Phòng này chứa hổ và biển báo phòng số 6 thì sai.

Anh tù nhân phải mở cửa phòng nào để được tha tội và được cưới công chúa.

5 Supervised Learning

5.1 Tinhs toán tham số và dự đoán

- **Training examples:**
  - $X = \{1, 3, 2, 4, 3, 6\}$;
  - $y = \{1.50, 2, 3.50, 2.50, 1, 1.50\}$
- **Require:**
  a) Calculate: β
  b) Predict: x = 2.50

5.2 House Price Prediction

- **Mô tả:** Sử dụng Linear Regression để dự đoán giá của các căn nhà dựa trên các yếu tố như diện tích, số phòng, vị trí, v.v.
- **Dữ liệu:** Tập dữ liệu có thể được thu thập từ các trang web bất động sản, từ cơ quan quản lý bất động sản hoặc từ các dịch vụ môi giới bất động sản. Dữ liệu này bao gồm các thông tin về các căn nhà như diện tích, số phòng, vị trí, tiện ích xung quanh và giá bán thực tế.

- **Yêu cầu:**
  1. **Tiền xử lý dữ liệu:** Đọc và khám phá dữ liệu. Xử lý dữ liệu thiếu, loại bỏ nhiễu, chuẩn hóa dữ liệu nếu cần. Chọn các đặc trưng phù hợp cho mô hình Linear Regression.
  2. **Chia tập dữ liệu:** Chia dữ liệu thành tập huấn luyện và tập kiểm tra.
  3. **Xây dựng mô hình Linear Regression:** Sử dụng tập dữ liệu huấn luyện để xây dựng một mô hình Linear Regression.
  4. **Đánh giá mô hình:** Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình Linear Regression trong việc dự đoán giá nhà.
  5. **Tinh chỉnh mô hình:** Thử nghiệm với các tham số khác nhau của Linear Regression như hệ số điều chỉnh (alpha), số lượng đặc trưng, v.v. để cải thiện hiệu suất của mô hình.
  6. **Dự đoán:** Sử dụng mô hình tốt nhất để dự đoán giá nhà cho các căn nhà mới.

Ghi chú:
Python, thư viện phổ biến để triển khai các mô hình hồi quy tuyến tính (linear regression) là scikit-learn. Thư viện này cung cấp một lớp LinearRegression để xây dựng và huấn luyện mô hình hồi quy tuyến tính.

5.1 Logistic Regression

5.3 Iris Flower Classification

- **Mô tả:** Sử dụng Logistic Regression để phân loại loài hoa Iris dựa trên các đặc điểm hình học của hoa như chiều dài và chiều rộng của cánh hoa và đài hoa.
- **Dữ liệu:** Tập dữ liệu Iris là một tập dữ liệu phổ biến trong Machine Learning, chứa thông tin về ba loài hoa Iris: Iris Setosa, Iris Versicolor và Iris Virginica. Mỗi loài có 50 mẫu, với các đặc điểm như chiều dài và chiều rộng của cánh hoa và đài hoa.
- **Yêu cầu:**
  1. Tiền xử lý dữ liệu: Đọc và khám phá dữ liệu. Kiểm tra phân bố của các lớp và đặc điểm của dữ liệu. Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra.
  2. Xây dựng mô hình Logistic Regression: Sử dụng tập dữ liệu huấn luyện để xây dựng một mô hình Logistic Regression.
  3. Đánh giá mô hình: Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình Logistic Regression. Sử dụng các metric như độ chính xác (accuracy), recall, precision, F1-score để đánh giá.
  4. Tinh chỉnh mô hình: Thử nghiệm với các tham số khác nhau của Logistic Regression như độ sâu cây, số lượng mẫu tối thiểu cho mỗi nút lá, v.v. để cải thiện hiệu suất của mô hình.
  5. Dự đoán: Sử dụng mô hình tốt nhất để dự đoán loài hoa Iris cho các mẫu trên tập dữ liệu kiểm tra.

Lưu ý:
Có thể sử dụng thư viện scikit-learn của Python để triển khai mô hình Logistic Regression và thực hiện các bước tiền xử lý và đánh giá mô hình. Tập dữ liệu Iris có sẵn trong thư viện scikit-learn, có thể sử dụng hàm load_iris() để tải tập dữ liệu này.

6 Naive Bayes Classifier

6.1 Text Classification

- **Mô tả:** Sử dụng Naive Bayes Classifier để phân loại các văn bản vào các danh mục khác nhau dựa trên nội dung của chúng. Chẳng hạn, phân loại các bài báo, email hoặc tweet vào các danh mục như ”Thể thao”, ”Chính trị”, ”Giải trí”, v.v.
- **Dữ liệu:** Tập dữ liệu có thể được thu thập từ các nguồn như trang tin tức, blog, hoặc từ các tập dữ liệu đã được gán nhãn. Dữ liệu này bao gồm các văn bản đã được phân loại vào các danh mục mong muốn.
- **Yêu cầu:**
  1. Tiền xử lý dữ liệu: Đọc và khám phá dữ liệu. Loại bỏ các ký tự không cần thiết, chuẩn hóa văn bản, và tách từng từ trong văn bản.

2. Rút trích đặc trưng: Tạo vectơ đặc trưng cho mỗi văn bản bằng cách sử dụng kỹ thuật như TF-IDF (Term Frequency-Inverse Document Frequency) để biểu diễn các từ trong văn bản.
3. Chia tập dữ liệu: Chia dữ liệu thành tập huấn luyện và tập kiểm tra.
4. Xây dựng mô hình Naive Bayes Classifier: Sử dụng tập dữ liệu huấn luyện để huấn luyện mô hình Naive Bayes Classifier.
5. Đánh giá mô hình: Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình trong việc phân loại văn bản vào các danh mục.
6. Tinh chỉnh mô hình: Thử nghiệm với các tham số khác nhau của Naive Bayes Classifier để cải thiện hiệu suất của mô hình.
7. Dự đoán: Sử dụng mô hình tốt nhất để dự đoán danh mục của các văn bản mới.

Lưu ý:
Việc tiền xử lý dữ liệu và rút trích đặc trưng là rất quan trọng để xây dựng một mô hình Naive Bayes Classifier hiệu quả cho việc phân loại văn bản vào các danh mục khác nhau.

Python, thư viện phổ biến cho Naive Bayes Classifier là scikit-learn. Scikit-learn cung cấp một loạt các thuật toán Naive Bayes, bao gồm Gaussian Naive Bayes, Multinomial Naive Bayes và Bernoulli Naive Bayes, phù hợp với các bài toán phân loại dữ liệu liên tục, đa lớp và nhị phân, tương ứng.

7 Decision Tree

7.1 Calculate and draw Decision Tree Regressor with max depth = 3:

1. $X = \{2, 1, 3, 5\}$; $y = \{1.5, 2.5, 3, 2\}$
2. $X = \{2.5, 1.5, 2, 3, 3.5\}$; $y = \{1, 2.5, 3, 2.5, 1.5\}$
3. $X = \{(2, 5), (1, 5), (2, 0), (0, 3), (3, 5)\}$; $y = \{1, 1.5, 3, 3.5, 1.5\}$
4. $X = \{(1, 2, 3), (0, 1, 2), (2, 4, 1)\}$; $y = \{1, 1.5, 2\}$

7.2 Xây dựng Cây quyết định dự đoán kết quả Esports

Mô tả:
Bạn có dữ liệu của 10 trận đấu gần đây của một đội tuyển. Hãy giúp đội tuyển xây dựng cây quyết định để dự đoán xem họ sẽ Thắng (**Win**) hay Thua (**Loss**) dựa trên 2 yếu tố: **First Blood** (Giành mạng đầu) và **Late Game** (Đội hình mạnh về cuối trận).

Yêu cầu:

1. **Training Data:**

Trận	First Blood (FB)	Late Game (LG)	Kết quả (Lớp)
1	Yes	Yes	**Win**
2	Yes	Yes	**Win**
3	Yes	No	**Win**
4	Yes	No	**Loss**
5	No	Yes	**Win**
6	No	Yes	**Win**
7	No	No	**Loss**
8	No	No	**Loss**
9	No	No	**Loss**
10	Yes	Yes	**Win**

2. **Các bước thực hiện (Yêu cầu sinh viên)**
   - Tổng số trận: 10 (6 Win, 4 Loss).
   - Công thức Entropy: $H(S) = -p_{win} \log_2(p_{win}) - p_{loss} \log_2(p_{loss})$
3. **Tính Information Gain (IG) cho từng đặc trưng**
   - **Đặc trưng FB**: Chia tập dữ liệu thành 2 nhóm: $FB = Yes$ (5 trận) và $FB = No$ (5 trận). Tính Entropy cho từng nhóm và tính $IG(S, FB)$.
   - **Đặc trưng LG:** Chia tập dữ liệu thành 2 nhóm: $LG = Yes$ (5 trận) và $LG = No$ (5 trận). Tính Entropy cho từng nhóm và tính $IG(S, LG)$.

4. **Chọn nút gốc (Root Node)**
    - Đặc trưng nào có Information Gain cao nhất sẽ được chọn làm nút gốc.
5. **Vẽ cây và lặp lại**
    - Sau khi chọn nút gốc, kiểm tra xem các nhánh đã **"thuần khiết"** (chỉ toàn **Win** hoặc toàn **Loss**) chưa. Nếu chưa, tiếp tục phân tách bằng đặc trưng còn lại.

7.3 Iris Flower Classification

- **Mô tả:** Sử dụng Decision Tree để phân loại loài hoa Iris dựa trên các đặc điểm hình học của hoa như chiều dài và chiều rộng của cánh hoa và đài hoa.
- **Dữ liệu:** Tập dữ liệu Iris là một tập dữ liệu phổ biến trong Machine Learning, chứa thông tin về ba loài hoa Iris: Iris Setosa, Iris Versicolor và Iris Virginica. Mỗi loài có 50 mẫu, với các đặc điểm như chiều dài và chiều rộng của cánh hoa và đài hoa.
- **Yêu cầu:**
    1. Tiền xử lý dữ liệu: Đọc và khám phá dữ liệu. Kiểm tra phân bố của các lớp và đặc điểm của dữ liệu. Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra.
    2. Xây dựng mô hình Decision Tree: Sử dụng tập dữ liệu huấn luyện để xây dựng một mô hình Decision Tree.
    3. Đánh giá mô hình: Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình Decision Tree. Sử dụng các metric như độ chính xác (accuracy), recall, precision, F1-score để đánh giá.
    4. Tinh chỉnh mô hình: Thử nghiệm với các tham số khác nhau của Decision Tree như độ sâu cây, số lượng mẫu tối thiểu cho mỗi nút lá, v.v. để cải thiện hiệu suất của mô hình.
    5. Dự đoán: Sử dụng mô hình tốt nhất để dự đoán loài hoa Iris cho các mẫu trên tập dữ liệu kiểm tra.

Lưu ý:
Có thể sử dụng thư viện scikit-learn của Python để triển khai mô hình Decision Tree và thực hiện các bước tiền xử lý và đánh giá mô hình. Tập dữ liệu Iris có sẵn của thư viện scikit-learn, có thể sử dụng hàm load_iris() để tải tập dữ liệu này.

7.4 Titanic Survival Prediction

- **Mô tả:** Trong thảm họa Titanic, có nhiều yếu tố ảnh hưởng đến khả năng sống sót của hành khách, bao gồm giới tính, tuổi tác, lớp vé, v.v. Trong bài tập này, chúng ta sẽ sử dụng Decision Tree để dự đoán xem một hành khách có sống sót hay không dựa trên các thông tin như giới tính, tuổi tác, lớp vé, và nơi lên tàu.
- **Dữ liệu:** Tập dữ liệu Titanic có sẵn và được sử dụng rộng rãi trong cộng đồng Machine Learning. Bạn có thể tìm thấy tập dữ liệu này trên nhiều trang web, bao gồm Kaggle. Tập dữ liệu này bao gồm các thông tin như: PassengerId, Survived (0: không sống sót, 1: sống sót), Pclass (lớp vé), Sex (giới tính), Age (tuổi tác), SibSp (số lượng anh chị em hoặc vợ/chồng đi cùng), Parch (số lượng bố mẹ hoặc con cái đi cùng), Fare (giá vé), Embarked (nơi lên tàu).
- **Yêu cầu:**
    1. Tiền xử lý dữ liệu: Đọc và khám phá dữ liệu. Xử lý dữ liệu thiếu, chọn các đặc trưng phù hợp cho mô hình Decision Tree.
    2. Xây dựng mô hình Decision Tree: Sử dụng tập dữ liệu huấn luyện để xây dựng một mô hình Decision Tree.
    3. Đánh giá mô hình: Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình Decision Tree.
    4. Tinh chỉnh mô hình: Thử nghiệm với các tham số khác nhau của Decision Tree như độ sâu cây, số lượng mẫu tối thiểu cho mỗi nút lá, v.v. để cải thiện hiệu suất của mô hình.
    5. Dự đoán: Sử dụng mô hình tốt nhất để dự đoán sống sót cho các hành khách trên tập dữ liệu kiểm tra.

Lưu ý:
Có thể sử dụng các thư viện Machine Learning như scikit-learn của Python để triển khai mô hình Decision Tree và thực hiện các bước tiền xử lý và đánh giá mô hình.

8 Nonparametric Models

8.1 K-Nearest Neighbors (KNN)

8.1 Dùng thuật toán k-NN cho bài toán hoa Iris và phân loại email Spam

8.2 Support Vector Machine

8.2 Phân loại hoa Iris bằng SVM

1. **Mục tiêu:**
   - Làm quen với bộ dữ liệu thực tế đầu tiên trong sự nghiệp Data Science.
   - Hiểu cách SVM phân chia ranh giới giữa các loài hoa dựa trên đặc điểm hình thái.
   - Quan sát trực quan sự khác biệt giữa Linear Kernel và Non-linear Kernel (RBF, Poly).
2. **Dataset:**
   - **Iris Dataset:** Bao gồm 150 mẫu hoa thuộc 3 loài: Setosa, Versicolor, và Virginica.
   - **4 Đặc trưng:** Chiều dài lá đài (Sepal length), Chiều rộng lá đài (Sepal width), Chiều dài cánh hoa (Petal length), Chiều rộng cánh hoa (Petal width).
3. **Pipeline:**
   - 3.1. **Khám phá và Trực quan hóa dữ liệu**
     - Sinh viên vẽ biểu đồ tán xạ (Scatter Plot) kết hợp giữa các cặp đặc trưng (ví dụ: Petal Length vs Petal Width).
     - **Nhận xét:** Loài Setosa thường tách biệt hoàn toàn, trong khi Versicolor và Virginica thường nằm sát hoặc hơi chồng lấn lên nhau. Điều này đặt ra thách thức cho việc tìm siêu phẳng.
   - 3.2. **Huấn luyện mô hình SVM với các Kernel khác nhau**
     Sinh viên thực hiện huấn luyện 3 mô hình SVM riêng biệt để so sánh:
     - **Linear Kernel:** Thử phân chia bằng những đường thẳng đơn giản.
     - **Polynomial Kernel (Bậc 3):** Sử dụng đường cong bậc cao.
     - **RBF Kernel:** Sử dụng ranh giới hình tròn/elip linh hoạt.
   - 3.3. **Đánh giá và So sánh**
     - Sử dụng Confusion Matrix để xem mô hình nhầm lẫn giữa loài hoa nào với loài hoa nào nhất.
     - Tính toán Accuracy Score cho từng loại Kernel.
   - 3.4. **Tinh chỉnh tham số C**
     - Thay đổi giá trị C (ví dụ: 0.1, 1, 10, 100).
     - **Nhiệm vụ:** Quan sát xem khi C rất lớn, ranh giới có cố gắng ”uốn lượn” để không bỏ sót điểm dữ liệu nào (Overfitting) hay không.
   - 3.5. **Yêu cầu báo cáo**
     - **Biểu đồ ranh giới:** Chụp ảnh màn hình ranh giới quyết định của 3 loại Kernel (nên chọn 2 đặc trưng quan trọng nhất để vẽ trên không gian 2D).
     - **Phân tích:** Tại sao loài hoa Setosa luôn đạt độ chính xác 100%, trong khi hai loài còn lại thì không? (Gợi ý: Dựa trên tính chất Linearly Separable - Phân tách tuyến tính).
     - **Kết luận:** Trong trường hợp dữ liệu Iris, Kernel nào cho kết quả tốt nhất và cân bằng nhất?

8.3 Phân loại Email Spam với SVM

1. **Mục tiêu:**
   - Hiểu quy trình xử lý ngôn ngữ tự nhiên (NLP) cơ bản.
   - Áp dụng thuật toán SVM để tìm siêu phẳng phân cách dữ liệu cao chiều.
   - Thử nghiệm và so sánh các loại Kernels (Linear, RBF, Polynomial) để tìm mô hình tốt nhất.
2. **Dataset:**
   - Sử dụng bộ dữ liệu SMS Spam Collection hoặc Enron Spam Dataset.
   - Mỗi mẫu dữ liệu bao gồm: Nội dung văn bản và Nhãn (Spam/Ham).
3. **Pipeline:**
   - 3.1. **Text Preprocessing**
     - Chuyển về chữ thường, loại bỏ ký tự đặc biệt, dấu câu.
     - Loại bỏ **Stopwords** (các từ vô nghĩa như ”and”, ”the”, ”is”...).

- **Stemming/Lemmatization:** Đưa từ về dạng gốc (ví dụ: ”running” -> ”run”).

3.2. **Trích xuất đặc trưng (Vectorization)**
Đây là bước quan trọng để SVM có thể hiểu được văn bản.
- Sử dụng mô hình **Bag of Words (BoW)** hoặc **TF-IDF** (Term Frequency - Inverse Document Frequency).
- Lúc này, mỗi email sẽ trở thành một vector có hàng ngàn chiều (mỗi chiều tương ứng với một từ).

3.3. **Huấn luyện mô hình SVM**
- Chia dữ liệu thành tập Train (80%) và Test (20%).
- Sử dụng thư viện (như Scikit-learn) hoặc công cụ (Orange/KNIME) để huấn luyện SVM.
- **Nhiệm vụ:** Phải thử nghiệm ít nhất 2 loại Kernel:
  i. Linear Kernel: Giả định ranh giới là đường thẳng.
  ii. RBF Kernel: Để xử lý các mối quan hệ phi tuyến phức tạp.

3.4. **Tối ưu hóa tham số (Hyperparameter Tuning)**
- Sử dụng **Grid Search** để tìm cặp tham số (C, γ) tốt nhất.
- **Câu hỏi:** Khi tăng C, mô hình có xu hướng Overfitting hay Underfitting? Giải thích dựa trên khái niệm ”Hard Margin” và ”Soft Margin”.

3.5. **Yêu cầu báo cáo**
- **Đánh giá mô hình:** Sử dụng **Confusion Matrix**, tính các chỉ số **Precision, Recall, và F1-Score**. Tại sao trong bài toán Spam, chúng ta lại sợ ”Precision thấp” (nhầm email quan trọng thành rác)?
- **So sánh Kernel:** Lập bảng so sánh độ chính xác giữa Linear Kernel và RBF Kernel.
- **Trực quan hóa:** Sử dụng kỹ thuật giảm chiều (như PCA đã học ở chương trước) để đưa dữ liệu email về 2D và vẽ ranh giới phân cách của SVM.

9 Ensemble Learning

9.1 Wisdom of the Crowd

9.1 Trò chơi thực nghiệm: ”Đoán số hạt đậu” (The Jelly Bean Experiment)

Đây là cách tốt nhất để minh họa cho sinh viên thấy tại sao nhiều người đoán sai cộng lại có thể ra một kết quả đúng đến kinh ngạc.

Cách thực hiện:

1. **Chuẩn bị:** Lấy một bức ảnh (hoặc một cái hũ thật) chứa rất nhiều vật thể (ví dụ: hạt đậu, đồng xu, hoặc số kẹo trong hũ). Giả sử số lượng chính xác là 1,245 hạt.
2. **Thu thập dữ liệu:** Yêu cầu mỗi sinh viên (tầm 40-50 em) ghi ra giấy con số mà các em đoán. Tuyệt đối không được thảo luận.
3. **Phân tích:**
   - Chọn ra một vài cá nhân (những ”chuyên gia” đoán gần đúng nhất). Sai số có thể vẫn ở mức 10-20
   - Tính trung bình cộng của toàn bộ các con số mà cả lớp đã đoán.
4. **Kết quả:** Thường thì con số trung bình của cả đám đông sẽ gần sát với đáp án thực tế hơn bất kỳ cá nhân nào trong lớp.

Bài học rút ra:

- Mỗi cá nhân có một sự sai lệch (bias) riêng.
- Khi kết hợp lại (Ensemble), các sai lệch trái chiều sẽ tự triệt tiêu lẫn nhau, để lại giá trị cốt lõi gần với sự thật nhất.

9.2 Thiết kế hệ thống Ensemble thực tế

- **Đề bài:** Xây dựng AI dự đoán đội vô địch giải Esports. Bạn có 3 mô hình:
  1. **SVM** (chuyên về dữ liệu kĩ năng người chơi).
  2. **Decision Tree** (chuyên về chiến thuật cấm/chọn).
  3. **k-NN** (chuyên về lịch sử đối đầu).
- **Yêu cầu:** Hãy mô tả cách bạn kết hợp 3 mô hình này bằng kỹ thuật Voting hoặc Stacking để có kết quả tốt hơn là dùng riêng lẻ.

9.2 Random Forest

9.3 Làm lại các bài tập trên theo thuật toán Random Forest

10 Unsupervised Learning

10.1 K-Means Algorithm

10.1 Chạy từng bước thuật toán K-means

- **Dữ liệu:**
  1D gồm các điểm: $\{2, 4, 10, 12, 3, 20, 30, 11\}$.
- **Yêu cầu:**
  1. Chọn $K = 2$. Khởi tạo tâm cụm $C_1 = 3$ và $C_2 = 15$.
  2. Thực hiện vòng lặp 1: Gán các điểm vào cụm gần nhất.
  3. Tính toán lại tâm cụm mới.
  4. Thực hiện vòng lặp 2 và xem kết quả thay đổi ra sao.

10.2 Chạy từng bước thuật toán K-means

- **Dữ liệu:**
  1D gồm các điểm: $\{2, 4, 10, 12, 3, 20, 30, 11\}$.
- **Yêu cầu:**
  1. Chọn $K = 2$. Khởi tạo tâm cụm $C_1 = 3$ và $C_2 = 15$.
  2. Thực hiện vòng lặp 1: Gán các điểm vào cụm gần nhất.
  3. Tính toán lại tâm cụm mới.
  4. Thực hiện vòng lặp 2 và xem kết quả thay đổi ra sao.

10.2 Principal Component Analysis

10.3 Hệ thống nén và tái tạo khuôn mặt (EigenFaces)

- **Mục tiêu bài tập:**
  - Hiểu cách biểu diễn hình ảnh dưới dạng ma trận dữ liệu cao chiều.
  - Áp dụng PCA để giảm chiều dữ liệu (từ hàng ngàn pixel xuống vài chục thành phần chính).
  - Đánh giá chất lượng ảnh tái tạo và hiểu về ”Tỉ lệ phương sai tích lũy”.
- **Tình huống:**
  Một công ty công nghệ muốn xây dựng hệ thống lưu trữ hồ sơ nhân viên. Thay vì lưu mỗi tấm ảnh gốc nặng 2MB, họ muốn dùng PCA để chỉ lưu trữ một ”mã số” cực ngắn (ví dụ 50 con số) mà vẫn khôi phục được khuôn mặt nhân viên khi cần.
- **Pipeline:**
  1. **Data Collection:** Sử dụng bộ dữ liệu khuôn mặt nổi tiếng LFW (Labeled Faces in the Wild) hoặc ORL Faces Database. Mỗi tấm ảnh có kích thước 64×64 pixels $\rightarrow$ Không gian gốc có 4096 chiều.
  2. **Preprocessing:**
     2.1. **Flatten:** Chuyển ảnh về dạng vector phẳng .
     2.2. **Standardization:** Trừ đi ”khuôn mặt trung bình” (Average Face) của toàn bộ tập dữ liệu
  3. **Thực hiện PCA:**
     3.1. Tính toán ma trận hiệp phương sai.
     3.2. Tìm các trị riêng (Eigenvalues) và véc-tơ riêng (Eigenvectors).

3.3. Sắp xếp các véc-tơ riêng theo thứ tự giảm dần của trị riêng.

4. **Trực quan hóa "Eigenfaces":**
   4.1. Lấy 10 véc-tơ riêng đầu tiên và chuyển ngược chúng về dạng hình ảnh 64×64.
   4.2. Câu hỏi: Tại sao những hình ảnh này trông giống như "những khuôn mặt bóng ma"? (Đây chính là các đặc trưng quan trọng nhất: hốc mắt, sống mũi, khung hàm).
5. **Nén và Tái tạo (Compression & Reconstruction):**
   5.1. Chiếu một tấm ảnh mới vào không gian PCA chỉ với k thành phần (ví dụ $k = 10, 50, 100$).
   5.2. Dùng các hệ số này để dựng lại ảnh gốc.
   5.3. So sánh ảnh gốc và ảnh tái tạo bằng mắt thường và bằng chỉ số MSE.
6. **Yêu cầu báo cáo:**
   6.1. **Biểu đồ Scree Plot:** Vẽ biểu đồ thể hiện tổng phương sai tích lũy. Cần bao nhiêu thành phần chính để giữ lại được 90% thông tin của khuôn mặt?
   6.2. **Đánh giá định lượng:** Khi tăng số lượng thành phần chính k, chất lượng ảnh thay đổi như thế nào? Vẽ biểu đồ đường thể hiện mối quan hệ giữa k và sai số tái tạo.
   6.3. **Ứng dụng mở rộng:** Giải thích cách dùng các thành phần chính này để thực hiện bài toán Nhận diện khuôn mặt (Face Recognition) bằng thuật toán k-NN.

11 Neural Networks

11.1 Rosenblatt's Perceptron

11.1 Mô phỏng cổng logic AND

- **Dữ liệu:**
  - $(0, 0) \to 0$
  - $(0, 1) \to 0$
  - $(1, 0) \to 0$
  - $(1, 1) \to 1$
- **Yêu cầu:** Cho trọng số $w_1 = 0.5, w_2 = 0.5$ và ngưỡng (threshold) là 0.7. Hãy tính đầu ra của Perceptron cho 4 trường hợp trên và kiểm tra xem nó có hoạt động đúng như cổng AND không.

11.2 Cập nhật trọng số bằng tay (Manual Update)

- **Tình huống:** Perceptron đang có $w_1 = 0.3, w_2 = -0.2$ và $b = 0.1$. Tốc độ học $\alpha = 0.1$.
- **Dữ liệu đầu vào:** $x = (1, 1)$, nhãn thực tế $target = 1$.
- **Yêu cầu:**
  1. Tính dự đoán hiện tại của Perceptron (dùng hàm Step).
  2. Nếu dự đoán sai, hãy tính toán bộ trọng số và bias mới.

11.3 Vẽ ranh giới quyết định (Decision Boundary)

- **Yêu cầu:** Với phương trình $0.4x_1 + 0.6x_2 - 0.5 = 0$.
  1. Hãy vẽ đường thẳng này trên hệ tọa độ Oxy.
  2. Chỉ ra vùng không gian mà Perceptron sẽ dự đoán là lớp 1 và vùng dự đoán là lớp 0.

11.4 Thử thách XOR

- **Yêu cầu:** Hãy lập bảng giá trị cho cổng XOR. Vẽ các điểm dữ liệu này lên hệ tọa độ và giải thích bằng hình học tại sao một đường thẳng duy nhất không thể phân tách được 2 lớp này. Đây là bài tập quan trọng để dẫn dắt sv sang phần **Multi-Layer Perceptron (MLP)**.

11.5 Ảnh hưởng của Learning Rate

- **Thảo luận:** Nếu α quá lớn (ví dụ $\alpha = 10$) hoặc quá nhỏ (ví dụ $\alpha = 0.000001$), quá trình học của Perceptron sẽ gặp vấn đề gì? Hãy lấy ví dụ về việc một người đi bộ quá bước hoặc bước quá ngắn khi muốn dừng lại đúng vạch kẻ đường.

11.2 Multilayer Networks

11.6 Thiết kế mạng cho bài toán XOR

- Hãy vẽ sơ đồ mạng MLP (số lớp ẩn, số neuron) có khả năng giải bài toán XOR. Giải thích tại sao một lớp ẩn lại giúp tạo ra ranh giới quyết định cong.

11.7 Tính toán Forward Pass

- Cho mạng **2-2-1** với các trọng số và bias cho trước. Hãy tính tay giá trị đầu ra tại lớp ẩn và lớp cuối khi đầu vào là $(0, 1)$.

11.8 So sánh hàm kích hoạt

- Tính thử giá trị đầu ra của một neuron khi dùng hàm Sigmoid và hàm ReLU. Thảo luận về ưu nhược điểm của từng hàm (ví dụ: ReLU tính toán nhanh hơn).

11.3 Backward Propagation of Errors

11.9 Phân loại Iris Dataset

- **Mô tả:** Sử dụng Rosenblatt's Perceptron để phân loại các loài hoa Iris dựa trên các đặc trưng của chúng.
- **Dữ liệu:** Tập dữ liệu Iris Dataset chứa các thông tin về kích thước cánh hoa và lá hoa của các loài Iris.
- **Yêu cầu:** Huấn luyện perceptron để phân loại các loài Iris thành các nhóm khác nhau dựa trên các đặc trưng đã cho.

11.10 Dự đoán Diabetic Patients

- **Mô tả:** Sử dụng Rosenblatt's Perceptron để dự đoán liệu một bệnh nhân có mắc bệnh tiểu đường hay không dựa trên các chỉ số sinh học.
- **Dữ liệu:** Tập dữ liệu chứa các thông tin về chỉ số sinh học của bệnh nhân và kết quả xác định có mắc tiểu đường hay không.
- **Yêu cầu:** Xây dựng và huấn luyện Perceptron để dự đoán khả năng mắc bệnh tiểu đường của một bệnh nhân dựa trên các chỉ số sinh học.

11.11 Phân loại Email Spam

- **Mô tả:** Sử dụng Perceptron để phân loại các email thành hai loại: spam và không phải spam, dựa trên nội dung của email.
- **Dữ liệu:** Tập dữ liệu chứa các email đã được gán nhãn là spam hoặc không spam.
- **Yêu cầu:** Xây dựng một perceptron để phân loại các email thành hai nhóm và đánh giá hiệu suất của mô hình.

Ghi chú:
Có thể sử dụng thư viện scikit-learn của Python để triển khai Rosenblatt's Perceptron. scikit-learn cung cấp một lớp Perceptron để xây dựng và huấn luyện mô hình Perceptron.

"For the things we have to learn before we can do them, we learn by doing them."
— Aristotle, *The Nicomachean Ethics* (Book II) —